

★ WEEK 5 · LECTURE

Classification & Our First Anomaly Detector

Logistic, KNN, Naive Bayes — and LOF for the odd one out

Expanded edition · 38 slides · with three hands-on labs

Last week we predicted numbers. This week we sort things into bins: yes / no, fraud / safe. We meet three classic classifiers, then our first anomaly detector.

Where We Are Going Today

Week 4 = predict a number with a budget. Week 5 = predict a category, and spot anomalies

1

What is classification?

Predicting a category (yes/no) instead of a number. Meet Logistic Regression first.

2

Three ways to classify

Logistic (line-drawer), KNN (neighbor-asker), Naive Bayes (the diagnosing doctor).

3

Catching the odd one out (LOF)

Our first anomaly detector — finds the loner with no labels at all.

By the end you will train all three classifiers and catch fraud with zero labels.

1

PART 1 · CLASSIFICATION

From Numbers to Categories

Classification and Logistic Regression

A Different Kind of Question

Regression predicts how much; classification predicts which bin. Most bank decisions are yes/no.

Regression vs. Classification — Two Kinds of Question

Regression (Week 4)

Predicts a NUMBER.

"How much will this stock return?"

"What price next month?"

Answer can be any value:

+3.2%, -0.5%, 142.7 ...

Output lives on a number line.

Classification (Week 5)

Sorts into a BIN (category).

"Will this loan default? yes / no"

"Is this transaction fraud? yes / no"

Answer is a label:

0 or 1, spam or not, up or down.

Output is a choice from a few options.

From a Banker's Gut to a Number

How the credit score replaced 'I have a good feeling about you'

"Before credit scores, a loan often depended on whether the banker liked your face. Math made it fairer — and far more accurate."

The old way

- A human judged each applicant by instinct.
- Slow, inconsistent, and easy to bias.
- Two bankers could reach opposite decisions.

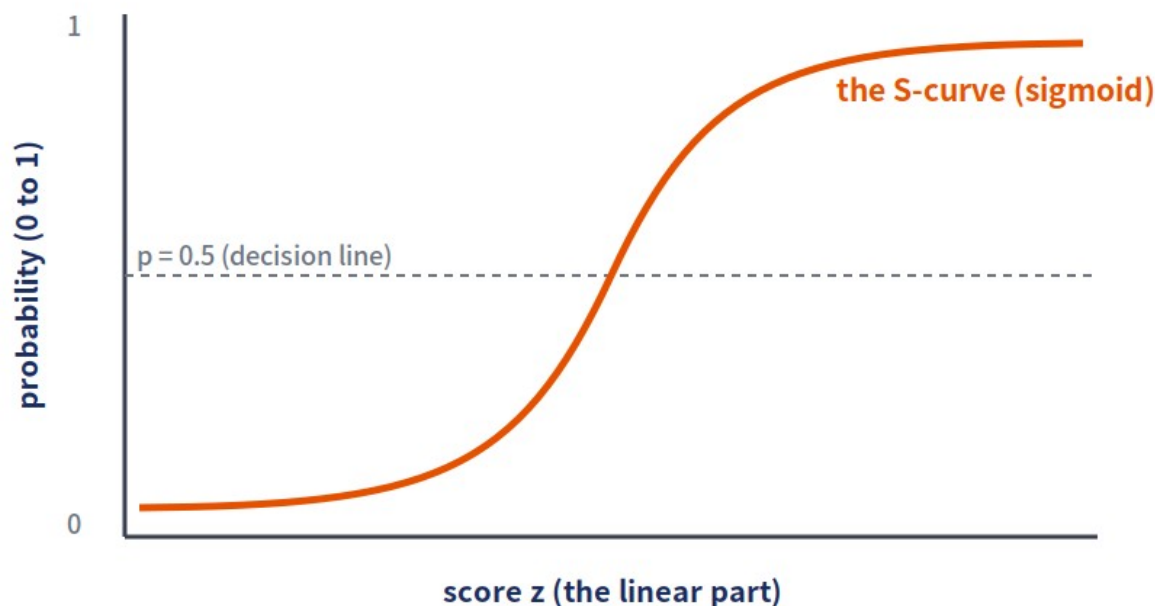
The ML way

- Feed past loans (features + repaid yes/no) to a model.
- It outputs a probability of default for each new applicant.
- Logistic Regression is the classic engine behind this.

Logistic Regression — the S-Curve

Take the straight line from Week 4 and squash it into a probability between 0 and 1.

Logistic Regression — Squash the Line into a Probability



Like a dimmer switch

A plain line can output any number, even -7 or +50. But a probability must stay between 0 and 1.

The S-curve gently squashes the line so the output always lands in $[0, 1]$.

Low score → near 0 ("no").

High score → near 1 ("yes").

Logistic Regression, as a Formula

A linear score z , then squashed by the S-curve

$$p = \frac{1}{1+e^{-z}}, \quad z = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

predict 1 if $p \geq 0.5$, else 0

Reading it

- z is Week 4's weighted sum of clues.
- The S-curve turns any z into a probability 0–1.
- If $p \geq 0.5$, predict 'yes'; otherwise 'no'.

Why it's loved

- Outputs a real probability — 'how sure?'.
- Readable weights: which clue raised the risk?
- Fast — a strong baseline for any yes/no task.

The S-Curve by Hand

Three scores in, three probabilities out — Week 1's risk score $z = 2 \cdot \text{Debt} - \text{Income}$ (full version: problem set)

Applicant (Week 1)	$z = 2 \cdot \text{Debt} - \text{Income}$	$e^{(-z)}$	$p = 1 / (1 + e^{(-z)})$	Read
A (debt-heavy)	+0.6	0.549	0.65	65% default risk
B (income-rich)	-1.0	2.718	0.27	27% — looks safe
G (the borderline)	-0.1	1.105	0.47	47% — a coin flip

$z = 0$ maps to exactly $p = 0.5$ — the S-curve pivots on the old decision line. G is Week 1's missed defaulter.

0.5 Is a Choice, Not a Law

The threshold is a business dial — move it and precision trades against recall

Threshold 0.5 (default)

- Balanced: flag only when default is more likely than not.
- Borderline G ($p = 0.47$) gets approved — and defaults.
- Exactly Week 1's expensive false negative.

Threshold 0.3 (cautious bank)

- Flag anyone with even a 30% risk — G is now caught.
- Recall rises: fewer missed defaulters.
- Precision falls: more good customers refused. The dial, not the model, sets the trade.

Same model, same probabilities — the threshold encodes what a mistake costs YOU.

2

PART 2 • THREE CLASSIFIERS

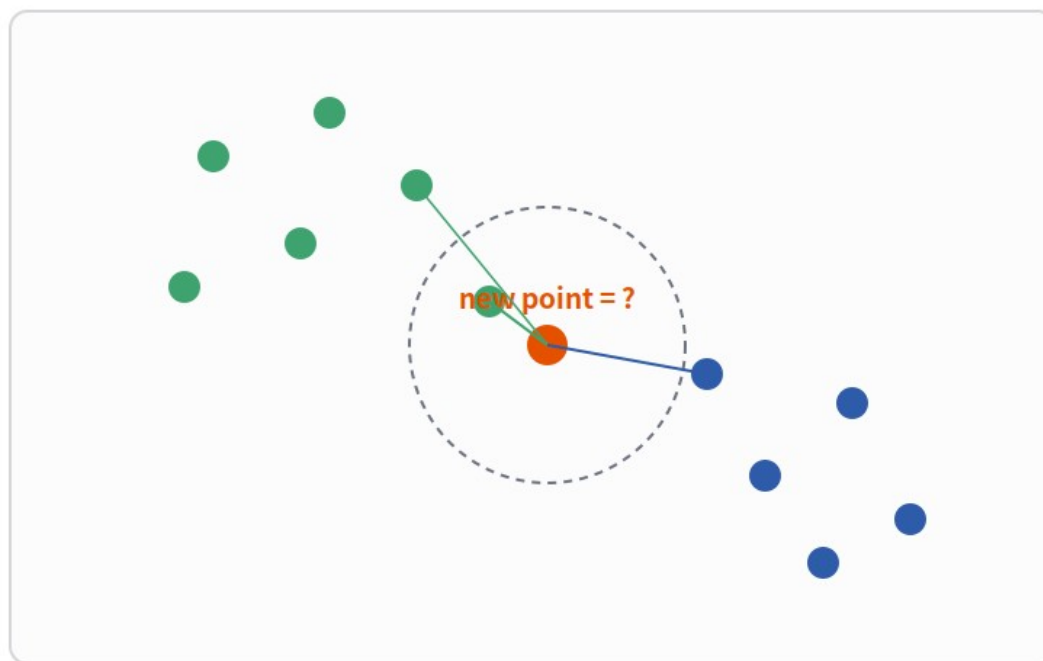
Three Ways to Classify

Logistic, KNN, and Naive Bayes

K-Nearest Neighbors — Ask the Neighbors

No formula to learn — just remember examples and let the closest ones vote. 'You are like your closest friends.'

K-Nearest Neighbors — Ask Your Closest Friends



How it decides

1. Find the k closest known points to the new one (here $k = 5$).
2. Let them VOTE on the label.
3. Majority wins → that's the answer.

"You are like your closest friends."

No training — it just remembers examples.

KNN — the Lazy but Clever Method

It does almost nothing until you ask it a question

To guess if a new customer will buy, find the 5 most similar past customers. If 4 of them bought, your best guess is: this one probably will too.

How k changes things

- Small k (e.g. 1): very sensitive, can chase noise.
- Large k: smoother, but may blur real groups.
- k is a knob you tune on the validation set.

Strengths & limits

- Strength: captures curvy, complex boundaries naturally.
- Limit: slow when data is huge (must scan everyone).
- Limit: struggles when there are very many features.

KNN by Hand — Six Neighbors, One Verdict

New applicant Q(income 1.1, debt 0.6). Distances already sorted (full version: problem set)

Neighbor	d^2 to Q	Defaulted?	k = 1 uses	k = 3 uses	k = 5 uses
A	0.02	Yes	✓	✓	✓
B	0.05	No		✓	✓
E	0.10	No		✓	✓
C	0.13	Yes			✓
D	0.17	No			✓

k = 1 says Default (1–0); k = 3 says Safe (1–2); k = 5 says Safe (2–3). The single nearest neighbor was noise.

Naive Bayes — the Diagnosing Doctor

Update your belief as each clue arrives

A doctor sees fever, then cough, then fatigue — and updates the likely diagnosis at each step. Naive Bayes does the same with clues, combining them by the rules of probability.

Why 'naive'?

- It assumes each clue is independent of the others.
- In reality clues overlap — so this is a simplification.
- Yet it works shockingly well, especially for text and fast screening.

Where it shines

- Spam filters (word clues).
- Real-time fraud screening (instant, tiny memory).
- Any case where speed matters more than perfection.

The Idea Behind It: Bayes' Rule

Flip 'clues given class' into 'class given clues'

$$P(\text{class} \mid \text{clues}) = \frac{P(\text{clues} \mid \text{class}) P(\text{class})}{P(\text{clues})}$$

Reading it

- Left: what we WANT — chance of a class given the clues we see.
- Right top: how likely those clues are IF the class were true.
- $P(\text{class})$: how common the class is to begin with (the 'prior').

In plain words

- Start with a baseline belief (priors).
- Each clue nudges the belief up or down.
- End with an updated probability for each class — pick the highest.

The Doctor by Hand — Watch the Belief Climb

A transaction shows three clues in a row; fraud starts at a 1% prior (full version: problem set)

Clue arrives	$P(\text{clue} \mid \text{fraud})$	$P(\text{clue} \mid \text{normal})$	Belief $P(\text{fraud} \mid \text{clues so far})$
— (prior)	—	—	1.0%
foreign transaction	0.60	0.05	10.8%
midnight timestamp	0.50	0.10	37.7%
unusually huge amount	0.70	0.10	80.9% → FLAG

Each clue multiplies both stories; the ratio is the belief. Note: two strong clues still weren't enough — base rates are stubborn.

Three Families, Side by Side

Discriminative, instance-based, generative — three styles of the same task. Let the validation set decide.

Three Ways to Classify — Same Goal, Different Styles

Logistic · the line-drawer

Draws a boundary directly.

Family: discriminative.

Learns one S-curve that splits yes from no.

- + Fast, gives clear probabilities.
- + Easy to interpret the weights.
- Assumes a fairly simple boundary.

KNN · the neighbor-asker

Remembers all examples.

Family: instance-based.

For a new point, asks the k nearest known points to vote.

- + No training; very intuitive.
- + Captures curvy boundaries.
- Slow & shaky with many features.

Naive Bayes · the doctor

Updates belief from clues.

Family: generative.

Uses Bayes' rule; treats each clue as independent ("naive").

- + Extremely fast, tiny memory.
- + Great for real-time screening.
- The independence assumption is rough.

The Three Classifiers at a Glance

Same question, three personalities — keep this table next to the picture

	Logistic (line-drawer)	KNN (neighbor-asker)	Naive Bayes (the doctor)
How it learns	Fits weights on clues	Memorizes; votes at question time	Counts clue frequencies per class
Speed	Fast train, instant predict	No training; SLOW predict	Instant both ways
Readability	Betas read like sentences	'Because these 5 look like you'	Which clue moved the belief
Watch out for	Only straight boundaries	Many features, big data	Correlated clues break 'naive'

There is no single best one — try all three and let the VALIDATION set decide (never the test set).

3

PART 3 · ANOMALIES

Catching the Odd One Out

LOF — our first anomaly detector

Fraud Is Different — It's Rare and Sneaky

Sometimes we have almost no examples of the thing we want to catch

Only about 1 in 1,000 transactions is fraud, and tomorrow's fraud may look brand-new. We can't always rely on labeled examples — so we hunt for things that simply look ODD.

The challenge

- Very few fraud examples to learn from.
- New fraud patterns appear all the time.
- Labels are slow and expensive to get.

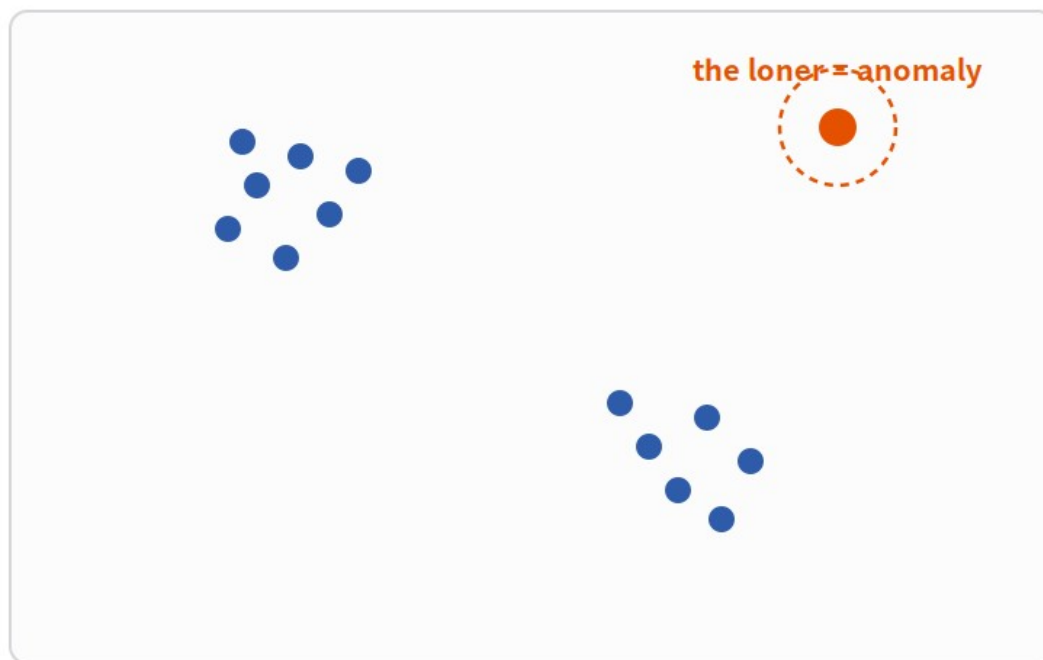
The idea: anomaly detection

- Don't learn 'fraud' directly.
- Instead, learn what NORMAL looks like.
- Flag anything that doesn't fit — no labels needed.

LOF — the Local Outlier Factor

Our first anomaly detector: it compares each point's crowding to its neighbors'. The loner scores highest.

LOF — Spotting the Loner in a Crowd



The big idea: density

At a party, most people stand in groups.

LOF asks each point: "are your neighbors packed close, or far away?"

A point in empty space (far from any group) gets a high anomaly score.

No labels needed — it finds the odd ones by how alone they are.

Why 'Local' Matters

LOF is smart about crowds of different densities

Some neighborhoods are naturally packed; others are spread out. LOF judges each point against its OWN local neighborhood — not one global rule.

How it scores

- Look at how tightly a point's neighbors are packed.
- Compare that to how packed THEY are around themselves.
- Much emptier than your neighbors → high anomaly score.

A picture in words

- A quiet villager is normal in a village, odd in Manhattan.
- A global rule ('flag anyone with < 3 neighbors') misjudges both.
- 'Local' means: compared to YOUR street, not the whole city.

Four Loner-Hunters, One per Week

LOF is the first of four anomaly detectors in this course — each sees 'odd' differently

W5 · LOF

Density — flags points sitting in emptier space than their neighbors

W6 · OCSVM

Boundary — draws a fence around 'normal' and flags whoever is outside

W7 · iForest

Isolation — the easier a point is to cut off, the odder it is

W8 · Autoencoder

Reconstruction — what the network can't redraw well is anomalous

Real fraud desks run several at once — different eyes catch different oddities.

Two Roads to Catching Fraud

With labels you learn 'fraud'; without labels you learn 'normal' — both matter

	Supervised (Logistic / NB / KNN)	Unsupervised (LOF & friends)
Needs labels?	Yes — past fraud marked by hand	No — raw transactions are enough
Catches NEW fraud?	Only patterns similar to the past	Yes — anything that just looks odd
Output	$P(\text{fraud})$ per transaction	An oddness score / flag
Typical role	The sharp scorer on known patterns	The backstop for the unknown

They are teammates, not rivals — the application slide shows exactly how a bank stacks them.

4

PART 4 · APPLICATION

Your Turn

Fraud detection: combine the tools

How a Bank Actually Combines These

No single model does it all — they work as a team

Stage 1 — fast screen

Naive Bayes

Runs on every transaction in real time. Instantly lets through the obviously-fine ones.

tiny memory, lightning fast

Stage 2 — sharper check

Logistic Regression

Scores the suspicious ones with a probability of fraud. Readable: which clue raised the alarm?

the explainable scorer

Stage 3 — the odd one out

LOF

Catches brand-new fraud with no past label — flags transactions unlike anything normal.

backstop for the unknown

Layered defense: fast screen → sharp score → anomaly backstop. Each tool covers the others' gaps.

Which Tool? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which tool?
Need a readable probability of default for a regulator	Logistic — betas read like sentences
Millions of transactions per second, instant screening	Naive Bayes — tiny memory, lightning fast
A weird new pattern with zero labeled examples	LOF — flags the loner, no labels needed
Small dataset, curvy boundary, interpretability optional	KNN — let the neighbors vote
The bank fears missed fraud more than false alarms	Lower the threshold — recall over precision

If you got 4 of 5, the toolbox of Week 5 is yours.

The Scorecard for Rare Classes

With 1-in-1,000 fraud, accuracy is a flattering liar — bring the honest metrics

The accuracy trap

- A model that always says 'safe' scores 99.9% accuracy.
- And catches exactly zero fraud.
- Lab 1 will show this number staring back at you.

What to report instead

- Precision: of the flags, how many were real fraud?
- Recall: of all fraud, how much did we catch?
- F1 (Week 3): high only when BOTH are high — the honest blend.

Rule for this course: on imbalanced data, never quote accuracy without F1 beside it.

5

PART 5 · HANDS ON

Your Turn — Hands On

Train three classifiers and catch fraud with zero labels

Getting This Week's Data

Companion notebook — classification data in Section 5 (offline) or 4 (Kaggle)

From the companion notebook

- Run Setup (§ 0).
- § 5 (offline) gives X, yb — a fraud-like labeled set that always works.
- § 4 shows the real Kaggle credit-card data (optional).

What you get back

- X: the clues (20 features per transaction).
- yb: the label (1 = fraud-like, 0 = normal), about 1% positive.
- Same data feeds both labs today — ready for scikit-learn.

[CODE] FML_Data_Setup_Guide.ipynb · § 5 offline classification set

```
from sklearn.datasets import make_classification
X, yb = make_classification(n_samples=20000, n_features=20, n_informative=6,
                           weights=[0.99, 0.01], random_state=42)
# X = clues, yb = label (about 1% positive, fraud-like)
```

Hands-On Lab 1 — Logistic vs. KNN vs. Naive Bayes

Train all three and compare on the same data

[CODE] colab_lab1.py · data from setup guide § 5

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score

Xtr, Xte, ytr, yte = train_test_split(X, yb, random_state=0)
models = [('Logistic', LogisticRegression(max_iter=1000)),
          ('KNN', KNeighborsClassifier()),
          ('NaiveBayes', GaussianNB())]
for name, m in models:
    m.fit(Xtr, ytr); pred = m.predict(Xte)
    print(name, 'acc', round(accuracy_score(yte, pred), 3),
          'F1', round(f1_score(yte, pred), 3))
```

Your task (step by step)

- Load X, yb (setup guide § 5).
- Split into train and test.
- Fit Logistic, KNN, and Naive Bayes.
- Compare their accuracy and F1.
- Q: which wins — and does accuracy alone mislead on rare fraud?

Hands-On Lab 2 — Find Fraud with LOF (No Labels)

Use only the clues; let LOF flag the loners

[CODE] colab_lab2.py · data from setup guide § 5

```
import numpy as np
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.01)
flag = lof.fit_predict(X)          # -1 = anomaly, 1 = normal
is_anom = (flag == -1)

# we trained WITHOUT labels; now check against yb
caught      = np.sum(is_anom & (yb == 1))
total_fraud = np.sum(yb == 1)
print(f'LOF flagged {is_anom.sum()} points')
print(f'caught {caught} of {total_fraud} real frauds,',
      'no labels used')
```

Your task (step by step)

- Use the same X (ignore the labels yb).
- Run LOF to flag ~1% as anomalies.
- NOW peek at yb to check.
- How many real frauds did LOF catch?
- Q: can an unlabeled method still find fraud?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 loop above, one plain-English sentence per line.
- 2) For the Logistic model, sweep the decision threshold from 0.1 to 0.9 and print a table:
threshold | precision | recall | F1.
- 3) Plot the precision-recall curve and mark the threshold with the best F1.
- 4) Build the bank's 3-stage pipeline from class:
NB screen -> Logistic score -> LOF backstop, and report total frauds caught vs. any single model.
- 5) In 3 sentences: why does lowering the threshold raise recall but hurt precision (Week 3 language)?

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 What does the S-curve (sigmoid) in Logistic Regression do?

- A** Speeds up training
- B** Squashes the score into a probability 0 to 1 ✓
- C** Deletes weak features
- D** Sorts the neighbors

Answer: The sigmoid maps any linear score z into a probability between 0 and 1 for yes/no decisions.

Q2 Why is LOF useful for fraud when we have no labels?

- A** It memorizes all fraud
- B** It flags points that sit alone (low local density) ✓
- C** It is the fastest model
- D** It needs Kaggle data

Answer: LOF scores how isolated each point is, flagging loners as anomalies — no labels required.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 Naive Bayes is 'naive' because it assumes:

- A** All classes are equally likely
- B** The clues are independent of each other ✓
- C** The data has no noise
- D** Fraud never changes

Answer: It multiplies clue probabilities as if clues never overlap — a simplification that works shockingly well.

Q4 You lower the fraud threshold from 0.5 to 0.3. Expect:

- A** Higher recall, lower precision ✓
- B** Higher precision, lower recall
- C** Both must rise
- D** Nothing changes

Answer: Flagging at 30% catches more real fraud (recall up) but also more false alarms (precision down).

Week 5 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
Classification	predict a bin (yes/no), not a number — most bank decisions.
Logistic	linear score z , squashed by the S-curve into $P(\text{yes})$; read the betas.
KNN	no training; the k nearest examples vote — k is the smoothing knob.
Naive Bayes	prior $\times$ clue likelihoods, clue by clue — the fast doctor.
LOF	flags points emptier than their own neighborhood — zero labels.
Threshold	0.5 is a choice; move it to trade precision against recall.

Anomaly detector #2 (OCSVM) arrives next week — same fraud, different eyes.

Four Tools, One Picture Each

Bonus — LOF (the loner detector): anomalies by low local density, with no labels at all

1

Logistic = the line-drawer

Squashes a linear score into a probability with the S-curve. Fast, readable, a great baseline.

2

KNN = ask your neighbors

Remembers examples; the k closest vote. Intuitive, captures curvy boundaries, slow on big data.

3

Naive Bayes = the doctor

Updates belief from clues via Bayes' rule. 'Naive' independence, but lightning fast for screening.

Keep the pictures: a line-drawer, voting neighbors, a diagnosing doctor — and one loner-hunter.

Week 5 — Three Things to Remember

Before next class: finish Labs 1–3 and note how many frauds LOF caught with no labels

1

Classification sorts into bins

Not a number but a category (yes/no). Logistic Regression squashes a line into a probability.

2

Three families, three styles

Logistic draws the boundary; KNN asks neighbors; Naive Bayes updates belief from clues. Try all three.

3

LOF finds the odd one out

Our first anomaly detector: flags low-density loners with no labels — perfect for rare, new fraud.

Next week — SVM: the widest possible safety margin (and OCSVM, anomaly detector #2).